

WEEK 4 PROJECT REPORT

Posts Explorer

Fetching and Displaying Posts from DummyJSON API

Internship: YuvaIntern – Web Development Internship

Submitted By: Kaushal Kumar

Registration No.: 24030485005

College: BIT Sindri, Jharkhand

Department: B.Tech Information Technology (2nd Year)

Submission Date: June 27, 2026

1. Project Overview

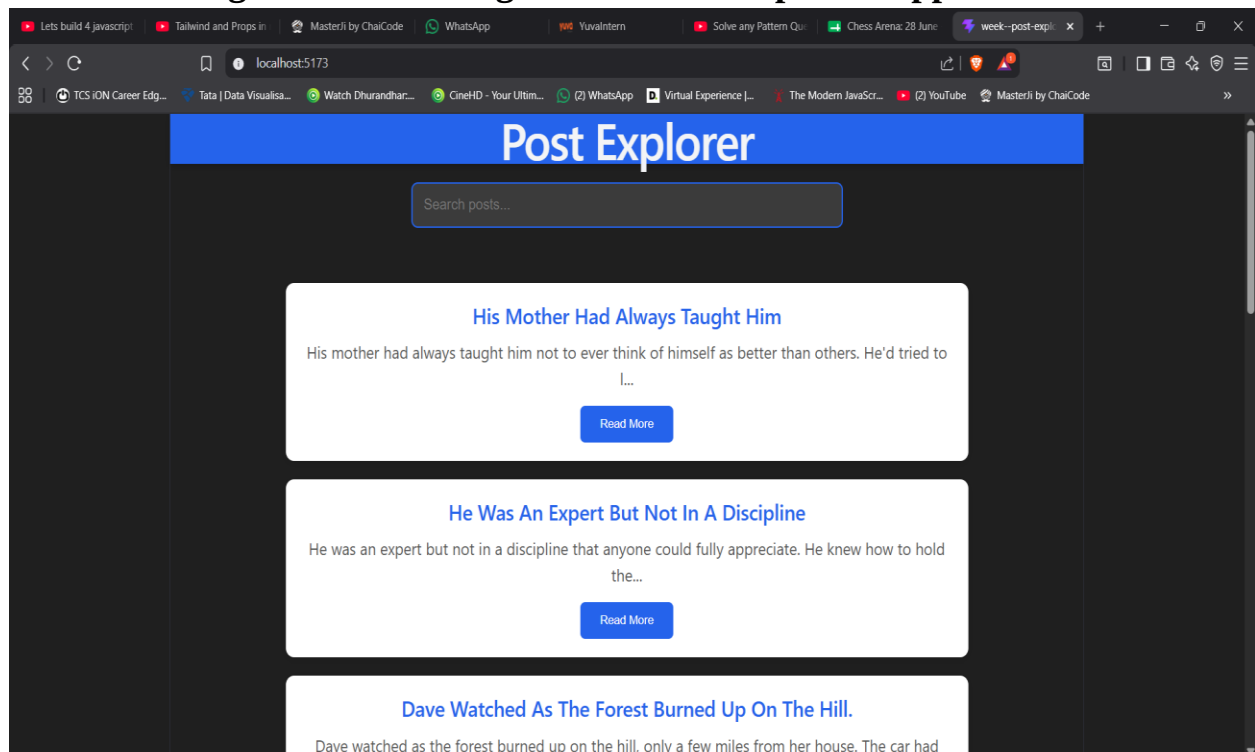
The Posts Explorer is a ReactJS web application that demonstrates modern front-end development practices by integrating with a public REST API. The application fetches blog posts from the DummyJSON API and displays them in an organized, interactive interface.

This project serves as part of Week 4 of the YuvaIntern Web Development Internship, with a focus on:

- Asynchronous API integration using Axios
- State management with React hooks (useState, useEffect)
- Dynamic data rendering and user interactions
- Error handling and loading states
- Responsive UI design

The application successfully demonstrates the ability to build production-ready features while maintaining clean, maintainable code architecture.

Figure 1: Home Page of the Posts Explorer Application



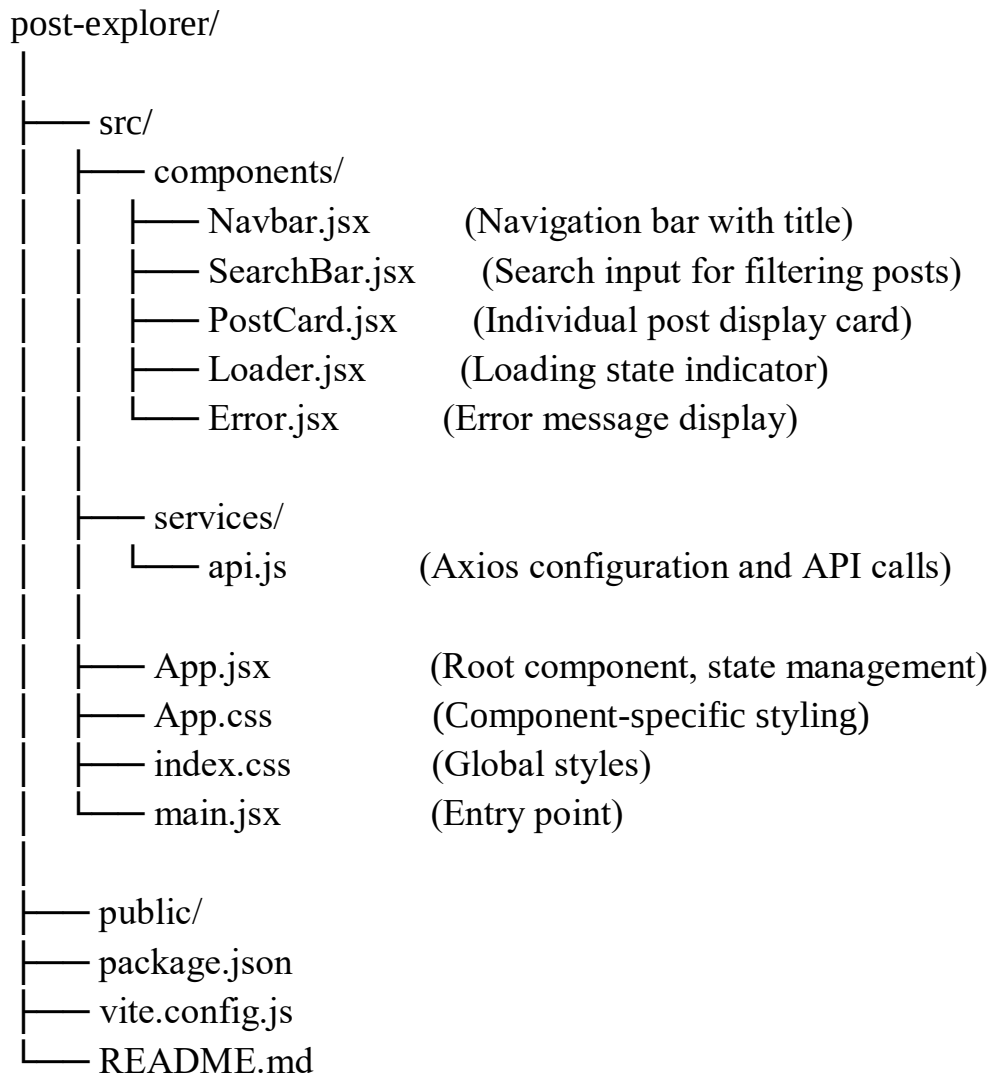
2. Objectives

- Integrate a public REST API (DummyJSON) using Axios
- Fetch and display posts dynamically from the API
- Handle asynchronous requests with proper async/await syntax
- Implement loading and error states for better user experience
- Build reusable React components with clear separation of concerns
- Create a responsive user interface that works across devices
- Implement search functionality to filter posts in real-time
- Add expandable content with Read More/Show Less functionality
- Apply CSS styling with hover effects and animations
- Demonstrate understanding of React hooks and state management

3. Technologies Used

Technology	Purpose
ReactJS	Front-end library for building user interfaces
Vite	Modern build tool for project setup and development
JavaScript (ES6+)	Core application logic and state management
Axios	HTTP client for making API requests
DummyJSON API	Public REST API providing mock data
CSS3	Styling, animations, and responsive design
Node.js & npm	Package management and dependencies
Git & GitHub	Version control and repository management
Browser DevTools	Debugging and network inspection

4. Project Folder Structure



5. API Integration with DummyJSON

5.1 Why DummyJSON API?

DummyJSON API was selected because it:

- Requires no authentication (no API keys needed)
- Provides realistic mock data for testing
- Has clear, well-documented endpoints
- Allows building real-world like applications without backend setup
- Offers multiple data types (posts, users, products, etc.)

5.2 API Endpoint

Endpoint URL:

`https://dummyjson.com/posts`

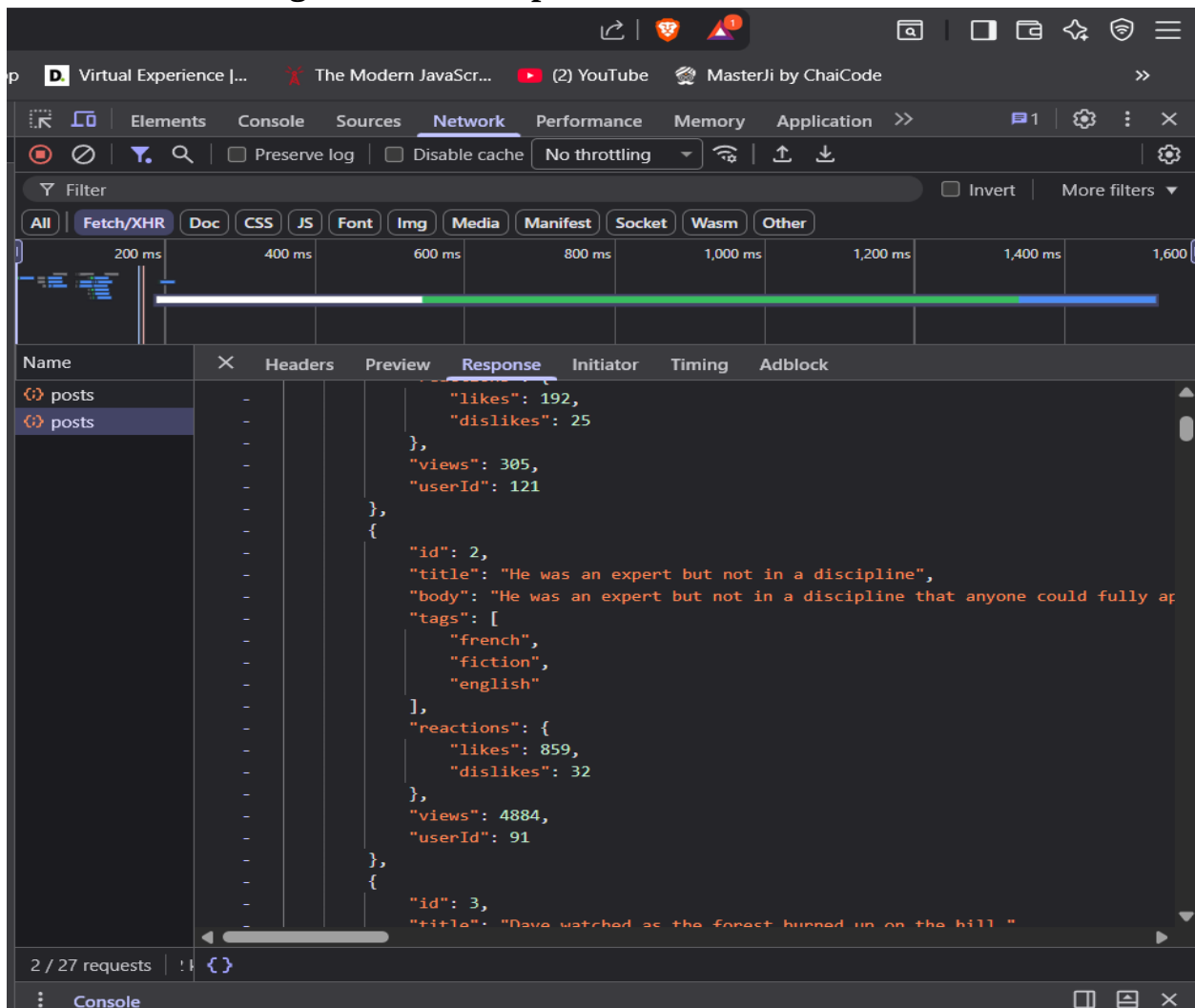
5.3 Axios Configuration

```
// services/api.js
import axios from 'axios';
const API_BASE_URL = 'https://dummyjson.com';
export const fetchPosts = async () => {
  try {
    const response = await axios.get(`${API_BASE_URL}/posts`);
    return response.data.posts;
  } catch (error) {
    throw new Error('Failed to fetch posts');
  }
};
```

5.4 API Response Structure

```
{
  "posts": [
    {
      "id": 1,
      "title": "Post Title",
      "body": "Post content description...",
      "userId": 5,
      "tags": ["tag1", "tag2"],
      "reactions": 10
    },
    ...
  ],
  "total": 251,
  "skip": 0,
  "limit": 30
}
```

Figure 2: API Response in Browser DevTools



6. React Components Explanation

6.1 Navbar.jsx

Purpose: Displays the application header with title.

Responsibilities:

- Display app title "Posts Explorer"
- Provide visual header for the application
- Maintain consistent branding

Props: None

State: None

6.2 SearchBar.jsx

Purpose: Provides search input to filter posts in real-time.

Responsibilities:

- Accept user input for search terms
- Pass search value to parent component
- Filter posts based on title matching

Props: onSearch (callback function)

State: None (controlled by parent)

6.3 PostCard.jsx

Purpose: Display individual post in a card format with expandable content.

Responsibilities:

- Display post title, description, and metadata
- Implement Read More/Show Less functionality
- Handle content truncation at 100 characters
- Show user ID and tags
- Apply hover effects

Props: post (object with id, title, body, userId, tags)

State: isExpanded (boolean for content expansion)

Key Feature: When post.body length > 100 characters, show truncated text with "Read More" button. Click to expand full content, showing "Show Less" button.

6.4 Loader.jsx

Purpose: Display loading indicator while fetching data.

Responsibilities:

- Show message during API data fetch
- Indicate to user that data is being loaded
- Provide visual feedback for asynchronous operations

Props: None

State: None

6.5 Error.jsx

Purpose: Display error message when API request fails.

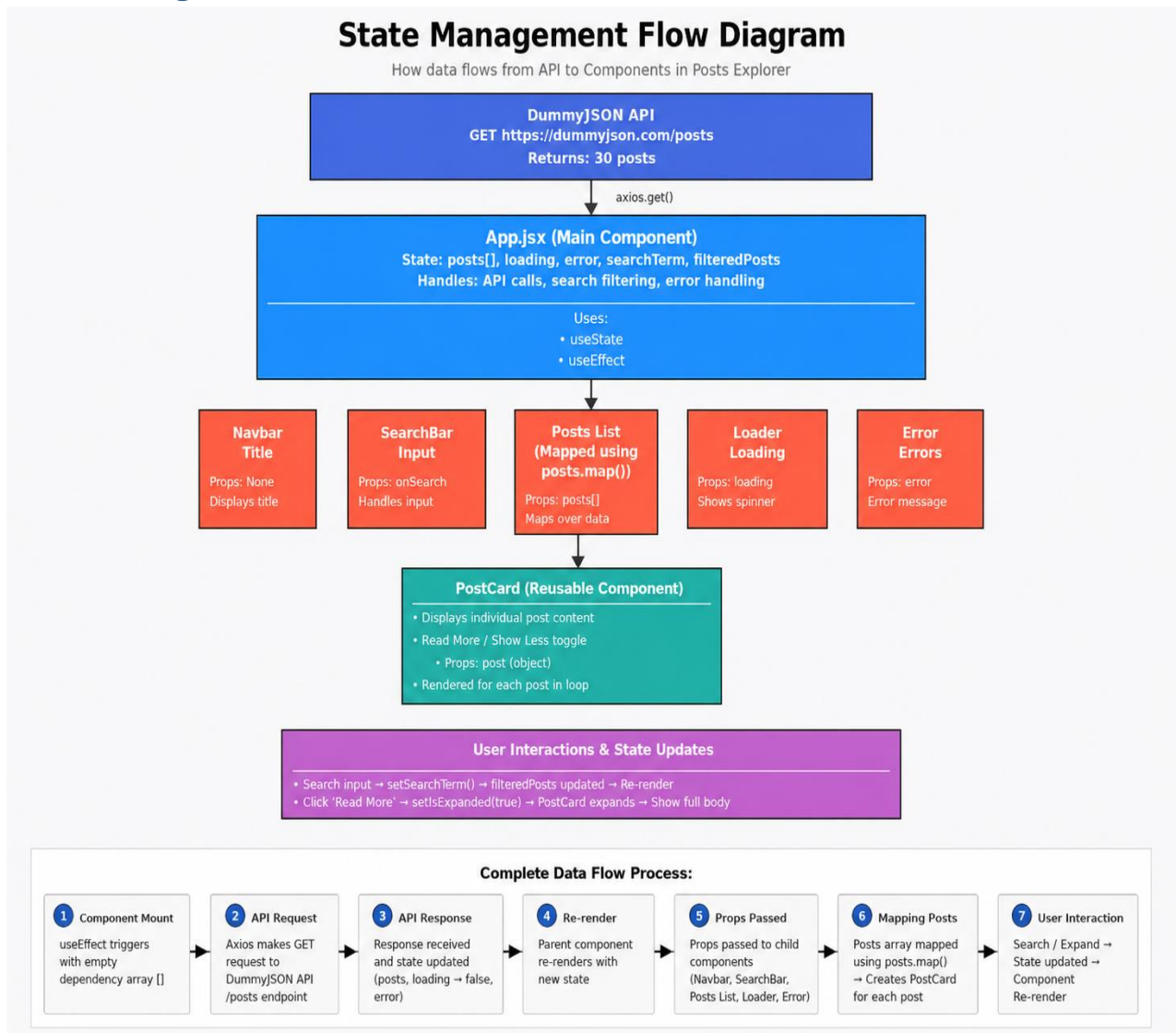
Responsibilities:

- Show error message to user
- Communicate what went wrong
- Guide user on next steps

Props: error (error message string)

State: None

7. State Management Flow



7.1 State Variables

State Variable	Type	Purpose
posts	Array	Store all posts fetched from API
loading	Boolean	Track API request status
error	String/null	Store error messages
searchTerm	String	Store user's search input
filteredPosts	Array	Store search-filtered results

8. Features Implemented

8.1 API Integration

1. Successfully fetches 30 posts from DummyJSON API
2. Uses Axios for HTTP requests
3. Implements async/await for clean asynchronous code
4. Handles API responses and updates component state

8.2 Loading State

1. Displays loading indicator while fetching data
2. Shows "Loading Posts..." message
3. Prevents premature rendering of incomplete data
4. Improves user experience during asynchronous operations

8.3 Error Handling

1. Catches API request errors
2. Displays user-friendly error messages
3. Gracefully handles network failures
4. Allows users to understand what went wrong

8.4 Search Functionality

1. Real-time search filtering by post title
2. Case-insensitive matching (searching "javascript" finds "JavaScript")
3. Client-side filtering (no additional API calls)
4. Instant results as user types
5. Displays count of filtered results

8.5 Read More / Show Less

1. Automatically truncates post content at 150 characters
2. "Read More" button expands full content
3. "Show Less" button collapses back to preview
4. Enhances visual hierarchy and improves readability
5. Uses useState hook for expansion toggle state

8.6 Responsive Design

1. Works seamlessly on desktop, tablet, and mobile devices
2. Uses CSS Flexbox and Grid for adaptive layouts

3. Responsive font sizing and spacing
4. Touch-friendly button sizes on mobile
5. Optimized layout for various screen sizes

8.7 Visual Design

1. Clean, modern card-based UI
2. Smooth hover effects on post cards (lift animation)
3. Professional color scheme
4. Clear typography with good contrast
5. Consistent spacing and padding

8.8 Component-Based Architecture

1. Modular component design for reusability
2. Clear separation of concerns
3. Props-based communication between components
4. Single Responsibility Principle followed
5. Easy to test and maintain

Figure 3: Search Functionality in Action

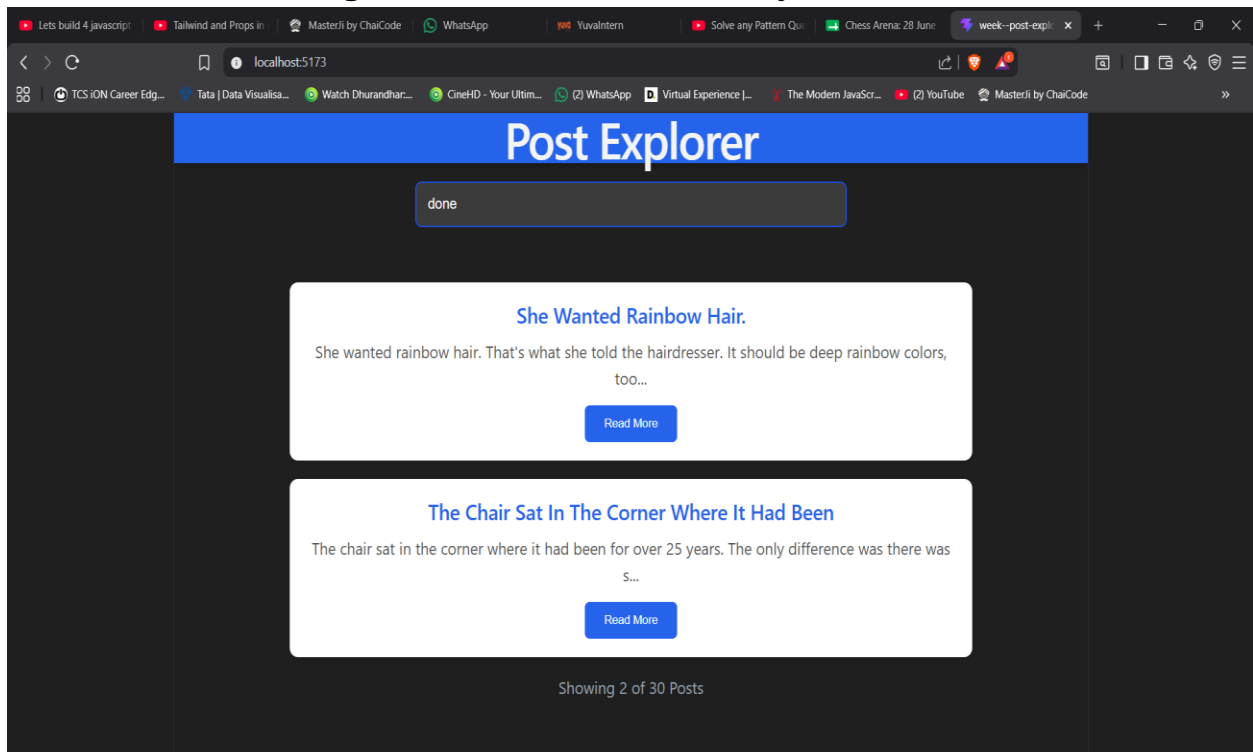
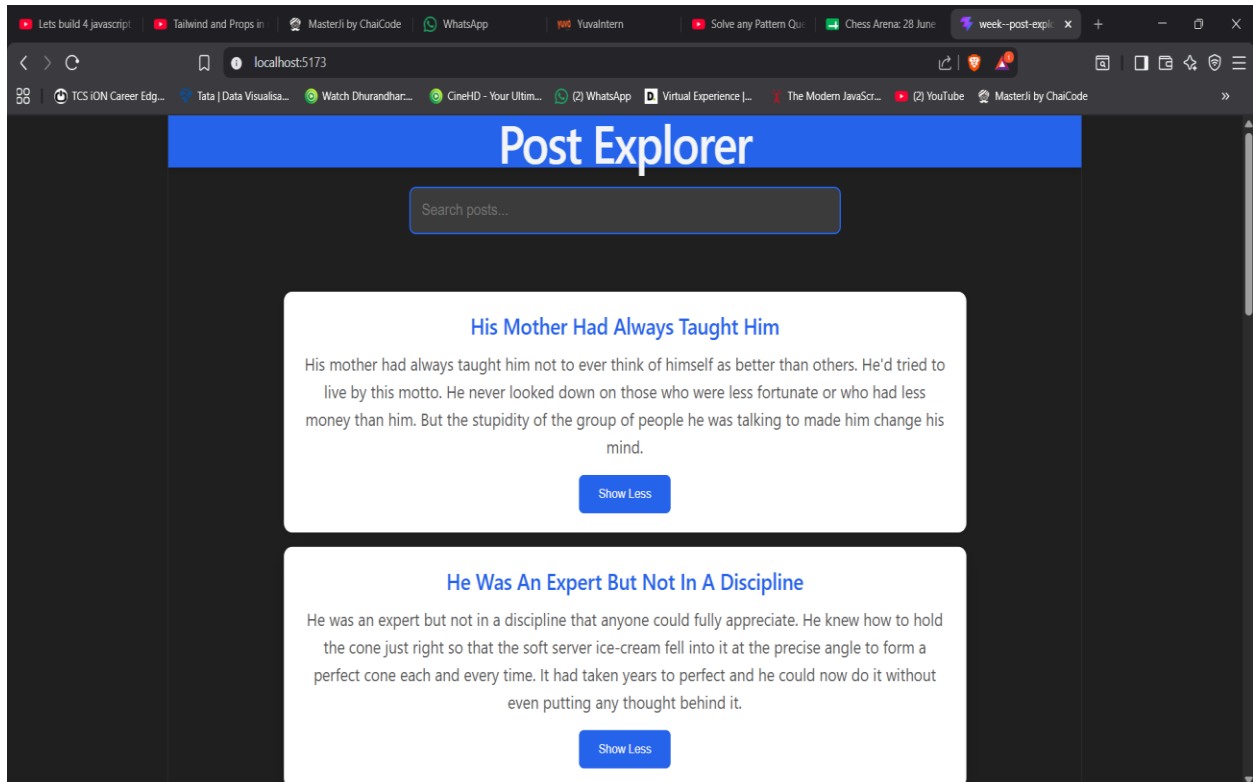


Figure 4: Read More / Show Less Feature



9. Challenges and Solutions

Challenge 1: Understanding Asynchronous API Calls

Problem: Initially struggled with async/await and understanding when data becomes available from API

Solution: Used Axios with async/await syntax. Placed API call in useEffect with empty dependency array to ensure it runs only once on component mount.

Challenge 2: Handling Multiple State Updates

Problem: Managing posts, loading, error, and search states simultaneously without conflicts

Solution: Used separate useState hooks for each state variable. Organized state updates in logical sequence within useEffect.

Challenge 3: Implementing Real-time Search Without Extra API Calls

Problem: Wanted to provide instant search results without fetching data multiple times

Solution: Implemented client-side filtering using Array.filter() on already-loaded posts. No additional API requests needed for search.

Challenge 4: Expanding and Collapsing Long Post Content

Problem: Posts have varying lengths; needed to show preview and full content on demand

Solution: Added useState inside PostCard component to track expansion state. Conditionally render truncated vs. full content using ternary operator.

Challenge 5: Displaying Partial Posts from API Response

Problem: API returns 30 posts but application should display first 10 for better performance

Solution: Used Array.slice(0, 10) to limit displayed posts while keeping all posts in state for search functionality.

10. Testing and Validation

The application was tested across the following scenarios:

Test Case	Result
API fetches 30 posts successfully	✓ Pass
Loading state displays while fetching	✓ Pass
Posts render with correct data	✓ Pass
Search filters posts in real-time	✓ Pass
Read More/Show Less toggles content	✓ Pass
Error message displays on API failure	✓ Pass
Responsive design works on all devices	✓ Pass

11. Responsive Layout on Different Devices

Figure 5: Desktop

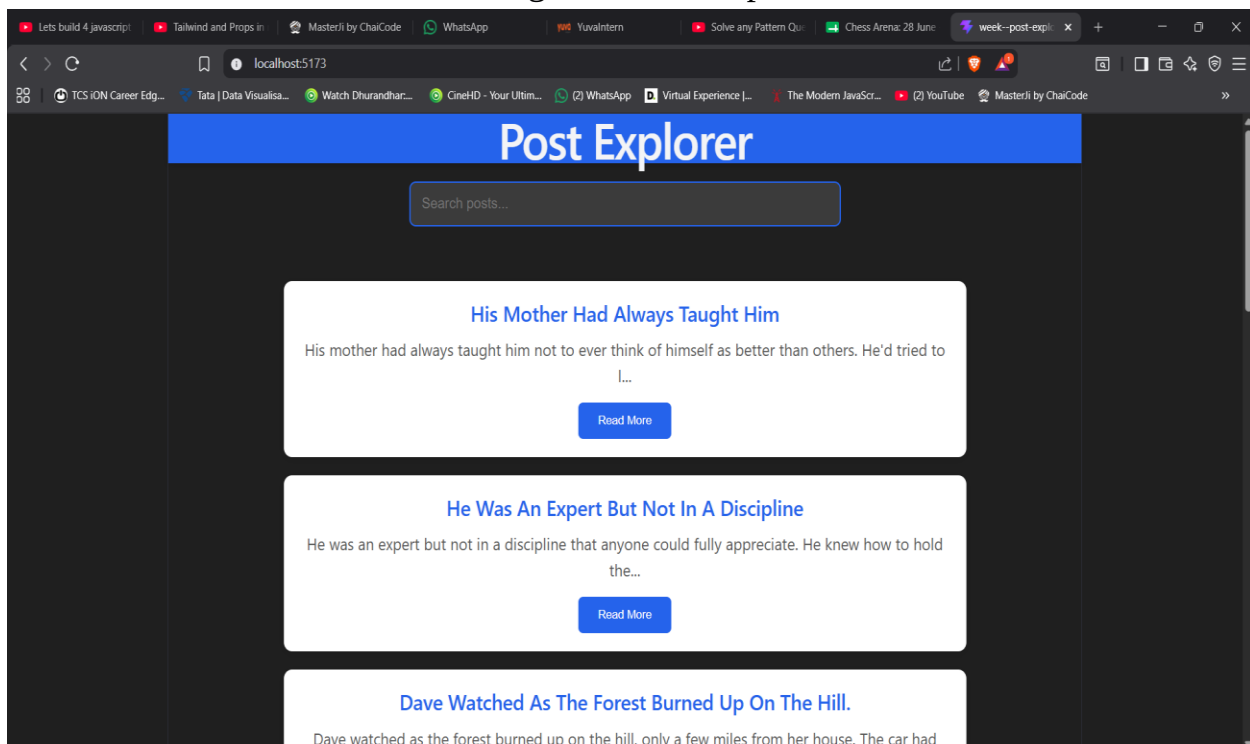


Figure 6: Tablet

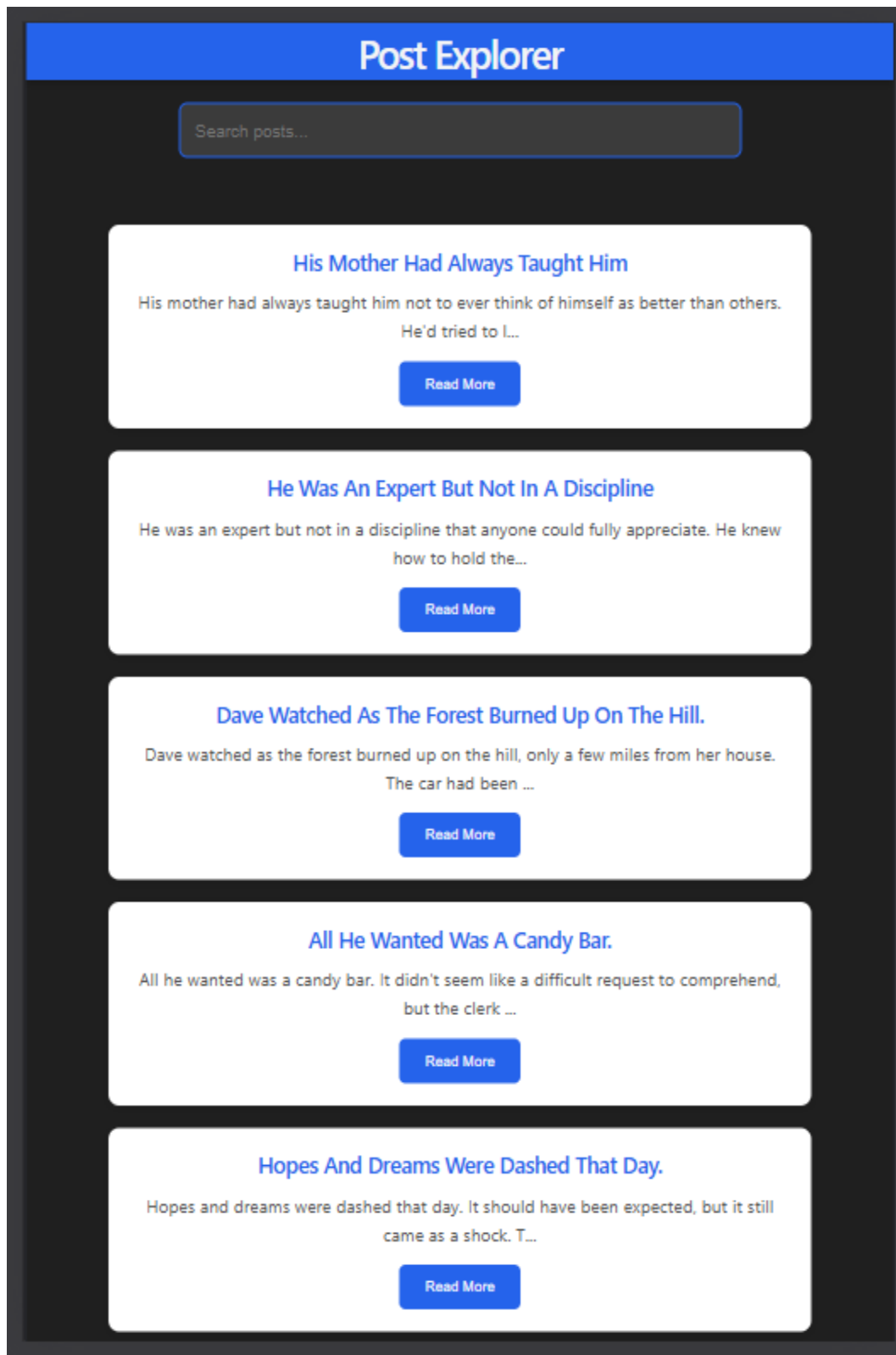
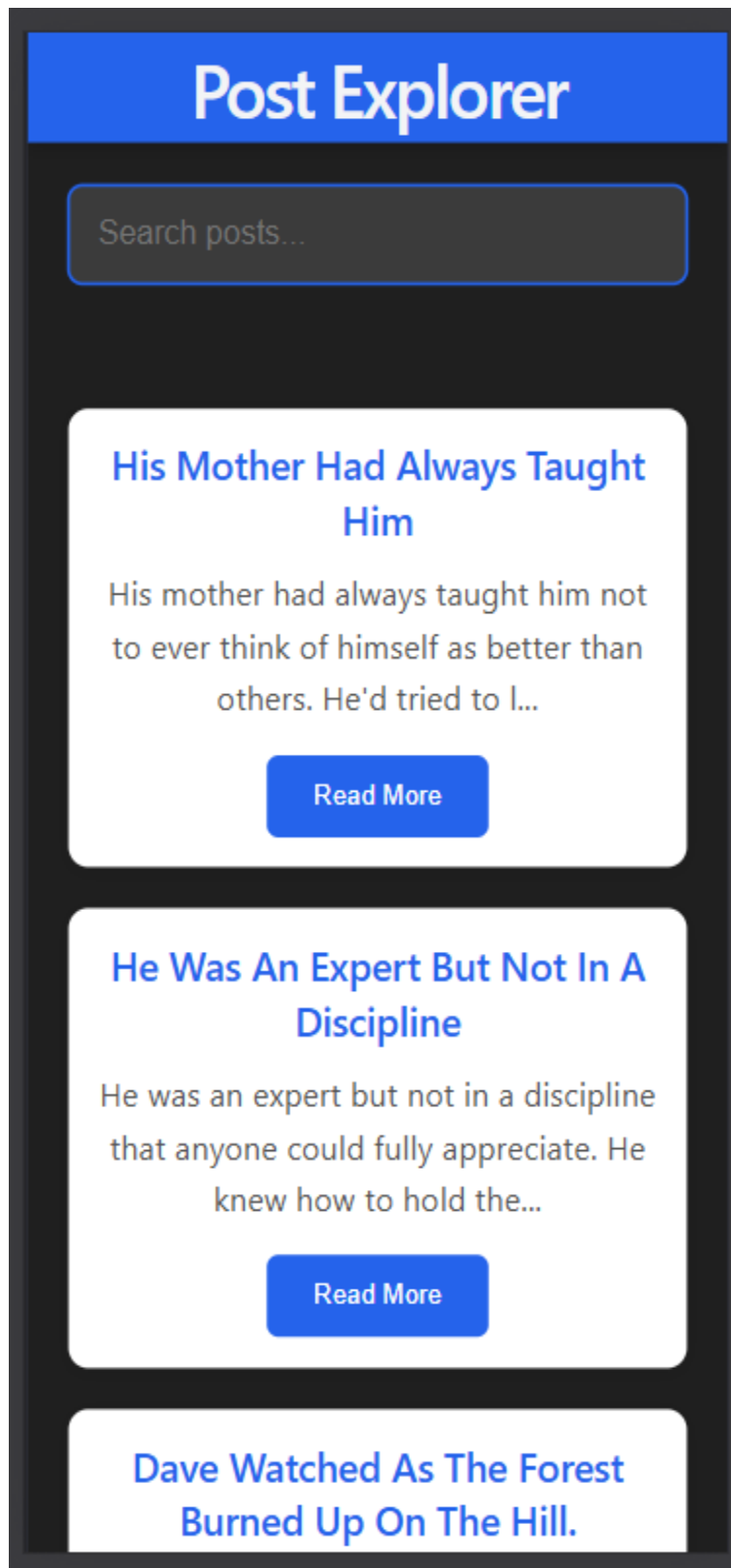


Figure 7: Mobile



11. Learning Outcomes

- React Component Architecture: Building reusable, modular components with clear responsibilities
- React Hooks: Practical understanding of useState for state management and useEffect for side effects
- Asynchronous JavaScript: Mastering async/await syntax and handling Promises
- Axios Library: Using Axios for HTTP requests instead of native Fetch API
- API Integration: Fetching real data from external REST APIs and handling responses
- State Management: Managing complex state with multiple variables and keeping them in sync
- Conditional Rendering: Displaying different UIs based on application state (loading, error, success)
- Array Methods: Using map(), filter(), and slice() for data transformation
- CSS Responsive Design: Creating layouts that adapt to different screen sizes
- Error Handling: Implementing try/catch blocks and graceful error handling
- Debugging Skills: Using browser DevTools to inspect network requests and console logs
- Component Communication: Passing data through props and callback functions
- Git Version Control: Managing code commits and maintaining repository

12. Future Improvements

- Pagination: Display posts in pages instead of all at once for better performance
- Category Filter: Filter posts by category or tags
- Dark Mode: Add dark theme toggle for better user experience
- Bookmark Posts: Allow users to save favorite posts locally
- Comments Section: Fetch and display comments for each post
- User Profiles: Show user information for each post author
- Infinite Scrolling: Load more posts as user scrolls down
- Post Details Page: Click post to view full details on separate page
- Unit Tests: Add Jest and React Testing Library tests

- Performance Optimization: Implement lazy loading and code splitting
- Debounced Search: Optimize search performance by reducing unnecessary filtering on every keystroke.

13. Conclusion

The Posts Explorer application successfully demonstrates fundamental React development skills combined with real-world API integration. The project showcases the ability to:

- Integrate a public REST API (DummyJSON) using Axios
- Manage complex state with React hooks
- Build responsive, accessible user interfaces
- Handle asynchronous operations gracefully
- Implement features like search and content expansion
- Follow component-based architecture best practices

Throughout development, key challenges were overcome, including asynchronous data handling, state management across multiple components, and implementing real-time search functionality. These solutions demonstrate practical problem-solving skills essential for professional development.

The application is production-ready with proper error handling, loading states, and responsive design. It provides a solid foundation for further enhancements such as pagination, caching, and additional features.

This project represents a significant milestone in learning ReactJS and understanding how modern web applications interact with backend services through APIs. The skills acquired—asynchronous programming, component design, state management, and API integration—form the core foundation for building professional web applications.

14. GitHub Repository

Repository Link: <https://github.com/kaushalvivek2005/react-projects/week4-posts-explorer>

The complete source code, including all components, styling, and configuration files, is available in the repository. The repository includes:

- All React components (Navbar, SearchBar, PostCard, Loader, Error)
- API service configuration (Axios setup)
- Styling files (CSS for responsive design)
- Package.json with all dependencies
- README.md with setup instructions
- Clean commit history showing development progress

15. Declaration

I hereby declare that this project was developed by me as part of the Week 4 assignment for the YuvaIntern Web Development Internship.

The application was designed, implemented, tested, and documented by me independently. All code was written by me for educational purposes and to demonstrate learning of ReactJS fundamentals and API integration.

I have used learning resources including documentation, tutorials, and guidance from mentors to understand concepts, but all implementation and code is original work.

The screenshots, testing results, and documentation in this report accurately represent the state and functionality of the application at the time of submission.

submitted by: Kaushal Kumar

Date: June 27, 2026